

Propuesta Técnica: Asistente RAG con Soberanía de Datos

Nombre: Erick Martin Peñafiel Picha

Arquitectura: Ollama + Qdrant (Self-Hosted)

1. Resumen Ejecutivo

Se propone el diseño de un sistema de **Generación Aumentada por Recuperación (RAG)** para la gestión del conocimiento técnico interno. La solución utiliza un stack *Open Source* que elimina la dependencia de proveedores de nube externos, garantiza latencias mínimas mediante infraestructura local y asegura que la propiedad intelectual de la empresa permanezca cifrada y protegida dentro de su propia red.

2. Stack Tecnológico de Grado Productivo

Componente	Tecnología	Justificación Técnica
Inference Engine	Ollama	Permite ejecutar LLMs (Llama 3.1, Mistral) con cuantización GGUF para optimizar el uso de VRAM en GPUs locales.
Vector Database	Qdrant	Base de datos vectorial nativa en Rust. Ofrece indexación HNSW (Hierarchical Navigable Small World) para búsquedas ultra-rápidas y filtrado complejo por metadatos (payloads).
Embeddings	nomic-embed-text	Modelo de alto rendimiento con una ventana de contexto de 8192 tokens, ideal para documentos técnicos densos.

Orquestación	LangChain / FastAPI	Provee la lógica de negocio, manejo de sesiones de usuario y streaming de respuestas.
--------------	---------------------	---

3. Arquitectura Detallada de Datos

A. Pipeline de Ingesta (ETL Vectorial)

Para que Qdrant sea eficiente, el proceso de carga sigue una jerarquía lógica:

1. **Carga de Documentos:** Procesamiento de archivos .md, .pdf y .docx mediante cargadores asíncronos.
2. **Transformación y Chunking:** * **Estrategia:** *Recursive Character Splitting*.
 - **Parámetros:** 700 tokens por fragmento con un *overlap* de 100 tokens.
 - **Lógica:** Se prioriza mantener los bloques de código y tablas íntegros para no romper la utilidad técnica.
3. **Vectorización:** Los fragmentos se pasan por el modelo de embedding en Ollama para generar vectores de 768 o 1024 dimensiones.
4. **Indexación en Qdrant:** Se insertan los vectores junto con un **Payload** (metadatos) que incluye: filename, hash_id (para evitar duplicados), department_access y timestamp.

B. Flujo de Recuperación (Retrieval)

Cuando el usuario lanza una consulta, el sistema realiza una **Búsqueda Híbrida**:

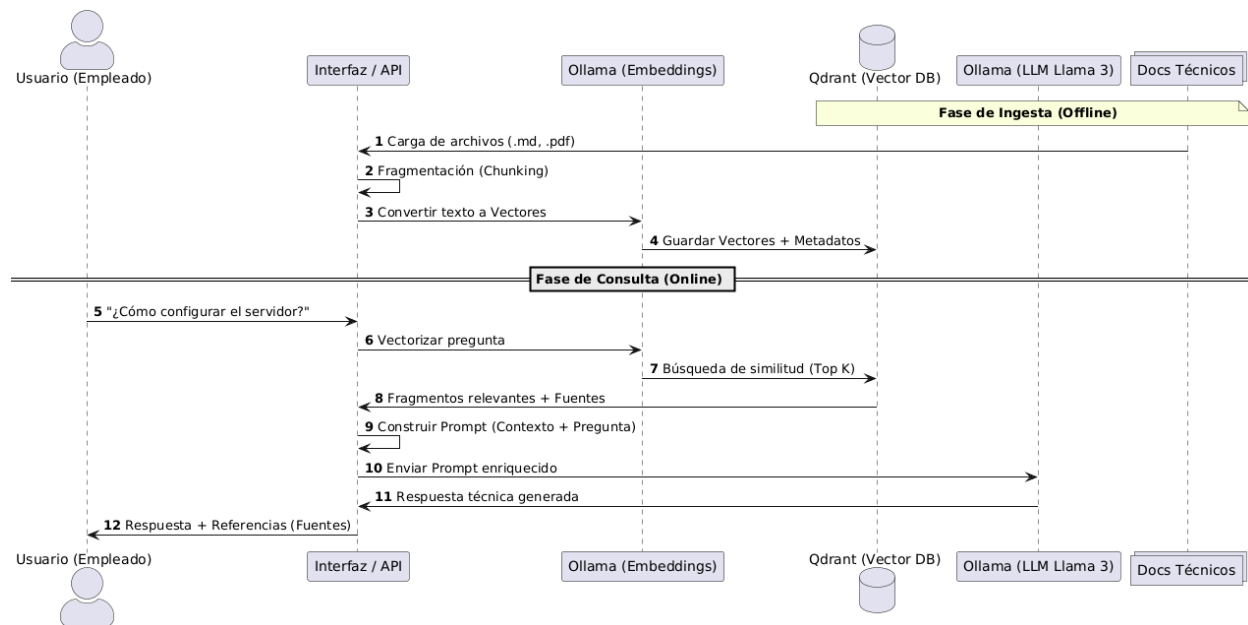
1. **Conversión de Query:** La pregunta se vectoriza en tiempo real.
2. **Búsqueda en Qdrant:** Se realiza una búsqueda de similitud de coseno.
3. **Filtrado por Metadatos:** Se aplican filtros booleanos en Qdrant (ej. filter: { "must": [{ "key": "status", "match": { "value": "verified" }}] }) para asegurar que la información no sea obsoleta.
4. **Reranking:** Se seleccionan los 5 fragmentos con mayor puntaje para construir el contexto.

4. Por qué Qdrant?

- **Eficiencia en Rust:** Qdrant está escrito en Rust, lo que garantiza un manejo de memoria eficiente y una latencia de búsqueda de milisegundos incluso con millones de vectores.
- **Filtrado de Payloads:** A diferencia de pgvector, Qdrant permite indexar los metadatos de forma independiente al vector, lo que hace que los filtros (por ejemplo, buscar solo en documentos de "Ciberseguridad") sean instantáneos.
- **Snapshot & Scalability:** Permite crear snapshots de la base de datos para migraciones calientes y soporta despliegues en clusters (Distributed Deployment) si el volumen de documentos crece exponencialmente.

5. Diagrama de Arquitectura (PlantUML)

Este diagrama representa la interacción entre los servicios Dockerizados:



6. Seguridad y Relevancia

Control de Alucinaciones y Relevancia

- **Groundness Check:** Implementar un paso de verificación donde el LLM evalúa si la respuesta generada está soportada 100% por el contexto recuperado.
- **Métricas RAGAS:** Evaluación automática de la fidelidad (*faithfulness*) y la relevancia de la respuesta.

Seguridad Multicapa

- **RBAC Integrado:** Qdrant filtrará los resultados en tiempo real basándose en el grupo de seguridad del usuario (LDAP/AD integration).
- **Sanitización de Inputs:** Protección contra *Prompt Injection* mediante capas de validación en FastAPI.

7. Ventajas y Limitaciones

Ventajas

- **Privacidad:** Cero fugas de datos hacia OpenAI o Anthropic.
- **Costo:** Sin facturación por tokens. Escalabilidad horizontal económica.
- **Consistencia:** El uso de Qdrant garantiza que las respuestas se basen en la "Única Fuente de Verdad" de la empresa.

Limitaciones

- **Hardware:** Requiere una inversión inicial en hardware (mínimo 24GB VRAM para modelos de 70B parámetros).
- **Mantenimiento:** Necesidad de gestionar backups de los volúmenes de Qdrant y actualizar los embeddings si se cambia el modelo base.

8. Estrategia de LLMOps: Monitoreo y Calidad

Para garantizar que el asistente pase de ser un prototipo a una herramienta de producción confiable, se implementará un ciclo de vida de **LLMOps (Large Language Model Operations)**:

A. Framework de Evaluación (RAGAS)

No basta con impresiones subjetivas. Se utilizará el framework **RAGAS** para auditar el sistema mediante tres métricas clave:

- **Fidelidad (Faithfulness):** Verifica que la respuesta del LLM provenga exclusivamente del contexto recuperado, eliminando alucinaciones.

- **Relevancia de Respuesta:** Mide qué tan útil es la respuesta para la duda específica del usuario.
- **Relevancia del Contexto:** Evalúa si el motor de búsqueda (Qdrant) está recuperando los fragmentos de documentos realmente necesarios.

B. Observabilidad y Feedback Loop

- **Trazabilidad:** Implementación de **LangFuse** o **Arize Phoenix** para visualizar el "camino" de cada consulta y detectar en qué paso (embedding, búsqueda o generación) se produce un error.
- **Refuerzo por Feedback:** La interfaz incluirá un sistema de votación (pulgar arriba/abajo). Las consultas con feedback negativo serán revisadas semanalmente para ajustar los *chunks* o re-entrenar el modelo de embeddings.